NEBIUS × NVIDIA GLOBAL AI HACKATHON · PROJECT REPORT

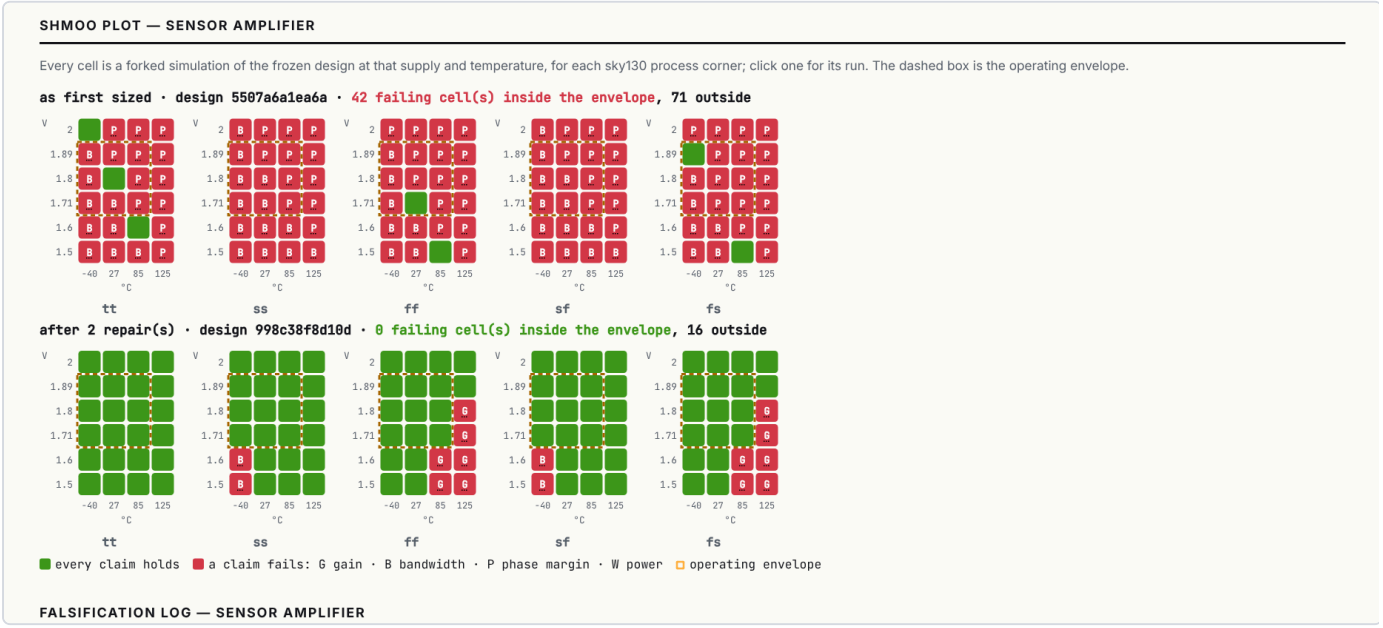
Tapeout

An AI chip designer that tries to prove its own datasheet wrong, and publishes where it breaks.

NVIDIA Nemotron (Ultra · Super · Nano) on Nebius Token Factory · Tavily · ngspice · SkyWater sky130

GitHub (public, MIT license): https://github.com/Amritha902/NEBIUS
Static demo (once GitHub Pages is enabled): https://amritha902.github.io/NEBIUS/demo/

3	42 → 0	399	413 / 413
red-team rounds to a design nothing in the envelope could break	failing shmoo cells inside the operating envelope, first sizing → repaired	real SPICE simulations in one tape-out (reference run)	recorded measurements reproduce on independent re-simulation



Shmoo plot from the reference run. Top: the design as first sized; red cells inside the dashed operating envelope are broken promises. Bottom: after two repairs.

Repository: github.com/Amritha902/NEBIUS (public; MIT, as detected by GitHub) · **Author:** Amritha S · **Date:** 30 October 2026
Status of evidence: every number in this report comes from a recorded run. The live Nebius Token Factory path is tested against a protocol-faithful mock but has **not yet run against the real service**; see §9.

Links

Source code	https://github.com/Amritha902/NEBIUS. Public repository. GitHub detects the license as MIT. Branches `main` and `claude/tender-archimedes-rs0ozn` are identical.
Static demo	https://amritha902.github.io/NEBIUS/demo/, live once GitHub Pages is enabled (Settings → Pages → `main`, `/docs`)
CI	https://github.com/Amritha902/NEBIUS/actions
Earlier work	https://github.com/Amritha902/cadence_auto, the circuit engine this builds on

Contents

1. Executive summary
2. The problem, and the idea
3. System architecture
4. Implementation, module by module
5. The falsification engine in detail
6. Development trial log: what was tried, what broke, what was fixed
7. Results: the reference run
8. Verification: tests, CI, clean room, mock live path
9. Limitations, and what is not yet verified
10. Running, deploying and demonstrating it
11. Appendix: commit history, file inventory, glossary

1. Executive summary

Tapeout turns a one-line product idea ("a heart-rate monitor front-end for a fitness band") into a mixed-signal chip datasheet. What sets it apart is what happens after design: a red team tries to break every claim on the datasheet before it ships.

- **Design team.**
  - Nemotron Nano and Tavily research the product.
  - Nemotron Ultra plans the silicon, gated by a deterministic compiler.
  - Nemotron Super races differential evolution to size each amplifier against real SkyWater 130 nm foundry models in ngspice.
  - Logic blocks are proved against their full truth table by transient simulation.
- **Red team.** Nemotron Ultra reads the sizing and margins and predicts the conditions where each claim should fail: process corner, supply, temperature, input common mode, load. A deterministic lab tests each prediction and bisects it to its exact boundary. A 45-point corner grid audits the predictions. Breaks inside the operating envelope go back to the design team. Breaks outside it are published as limits.
- **Arms race.** Each repair is attacked again. Every counterexample becomes a permanent regression test.
- **Output.** The datasheet carries:
  - a robustness envelope
  - a falsification log
  - a shmoo plot before and after repair

- a Tavily-verified commercial reference
- a provenance table showing which model did what

Every number links to the exact SPICE deck and ngspice log that produced it.

Reference run, heart-rate example:

- Round 1 broke phase margin (worst 23.2° against a 55° claim) and bandwidth (worst 76 kHz against 2 MHz).
- Repair 1 fixed those. Round 2's corner grid caught two failures the hypotheses missed (gain 59.7 dB, PM 54.6°).
- Repair 2 sized against all four corners that had ever broken a claim.
- Round 3 found nothing left to break inside the envelope.
- 399 simulations in 252 s.

2. The problem, and the idea

LLM agents can now draft analog circuits. Most demos, however, show a design that passes at one operating point: typical silicon, nominal supply, room temperature. A real chip is judged at its corners.

A generated datasheet is therefore a claim, not evidence. And the most valuable page of a real datasheet, the limits, is exactly the page generators leave out.

The idea: make failure the product. Instead of retrying until the simulator says "pass" and hiding the attempts, search for where each claim fails, prove it with simulation, repair what breaks inside the promised operating range, and publish the boundary of everything else, with the trace attached.

Design principles

1. **No model writes a netlist.** Models choose blocks from a fixed menu, propose numbers and hypotheses, and write prose. Synthesis is deterministic, so a wrong answer is a wrong *choice*, never a broken circuit.
2. **The simulator is the only judge.** A claimed failure counts only if a simulation shows it. A claimed pass counts only if one shows that.
3. **Every stand-in is declared.** If a model did not do something, the page says so.
4. **Every number is traceable.** Each one carries the run id of the deck that produced it.

3. System architecture

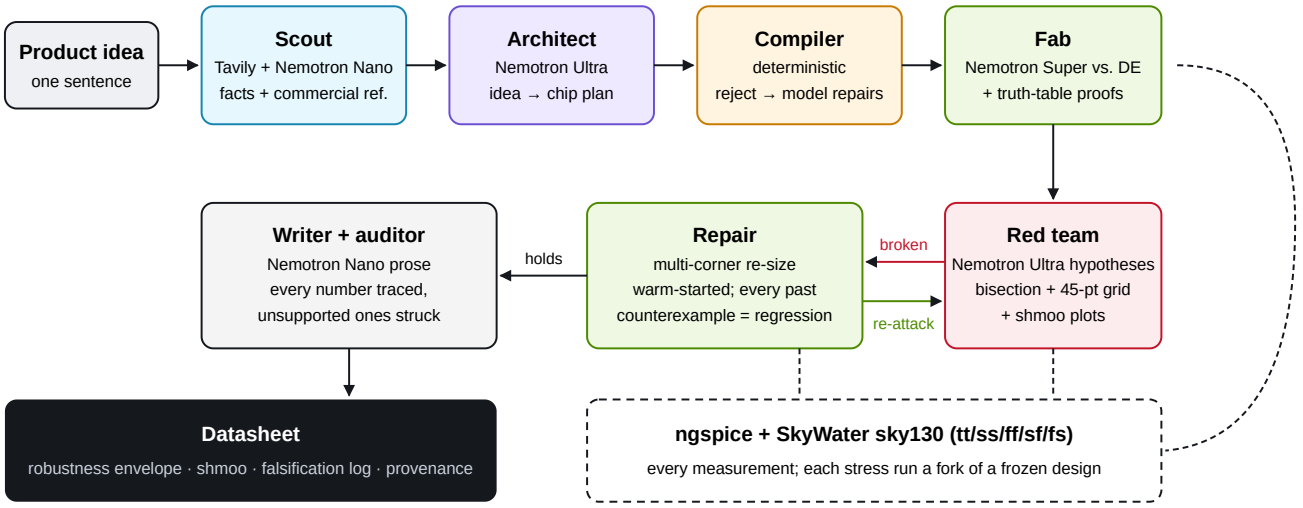


Figure 1. Pipeline. Solid arrows are the run order; dashed lines are simulator calls. Everything streams live to the browser as server-sent events.

Agents and models

Agent	Runs on	Job	Guard against hallucination
Scout	Tavily + Nemotron **Nano**	Search queries; numeric requirements from results; a commercial op-amp reference	A fact whose URL was not actually retrieved is dropped. A reference figure must appear *verbatim* in the retrieved page and carry the right unit.
Architect	Nemotron **Ultra** (Super repairs)	Product → blocks: amplifier specs with units, plus logic cells from a menu of 20	The deterministic compiler rejects infeasible or unparseable blocks; the rejection goes back to the model (up to 2 repair rounds)
Fab	Nemotron **Super** vs. differential evolution	Size each amplifier against ngspice; first to meet spec stops the other	Only simulated numbers count. The Nemotron lane is capped at 2× its simulation budget in calls.
Red team	Nemotron **Ultra** + deterministic lab	4–6 failure hypotheses; after each repair, 2–3 new ones targeting what changed	Hypotheses are clipped to legal knob ranges; a failure counts only if simulated; the corner grid audits the hypotheses
Writer	Nemotron **Nano**	Datasheet prose	The auditor matches every quantity and transistor count against simulated or specified values. Unsupported figures go back once, then are struck through.

Why this tier split.

- **Ultra** makes the two decisions where reasoning depth changes the outcome: the chip plan and the failure hypotheses.
- **Super** runs the latency-bound inner loop with the simulator, 10–20 calls per block, with thinking disabled for short JSON replies.
- **Nano** handles extraction and prose.

Model ids are discovered from Token Factory's `/v1/models` at startup and matched case-insensitively. They can be pinned with `TAPEOUT_MODEL_ULTRA`, `_SUPER` and `_NANO`.

4. Implementation, module by module

File	Lines	What it does
`tapeout/llm.py`	338	Token Factory client: streamed OpenAI-compatible chat, reasoning capture (`reasoning_content` deltas or inline `<think>` tags split across chunks), dropped-stream detection with retry, tier discovery, per-tier token/cost ledger
`tapeout/research.py`	253	Scout: Tavily search, Nano fact extraction with citation check, commercial-reference extraction with verbatim verification
`tapeout/architect.py`	317	Ultra planner, the deterministic compiler (spec parsing, measured gain/GBW/load envelope, cell menu), and the self-repair loop
`tapeout/fab.py`	491	Nemotron-vs-DE sizing race, single-stage → two-stage escalation, multi-condition evaluator, warm-started repair, logic truth-table proofs
`tapeout/falsify.py`	577	Red team: conditions and envelope, frozen-design forks, hypothesis proposer, bisection, corner grid, robustness table, breaking-condition selection, shmoo plots
`tapeout/writer.py`	217	Facts sheet, Nano prose, numeric auditor
`tapeout/pipeline.py`	310	One-click orchestration as an event stream; red-team campaign with regression accumulation; provenance
`tapeout/check.py`	126	`python -m tapeout.check [--offline]` : simulator, PDK, model resolution, one call per tier, Tavily
`tapeout/cli.py`	91	Terminal runner, `--json` result, `--record` for replays
`web/server.py`	162	FastAPI: live run as SSE, replay listing and time-compressed replay, status, health
`web/static/index.html`	657	Single-page UI: agent console, race chart, red-team board, shmoo, datasheet, trace modal, downloads
`scripts/export_static.py`	104	Self-contained replay page for GitHub Pages (no server, no API calls)
`bias/ (18 files)`	4,521	Circuit engine from the earlier cadence_auto project: spec parser, topologies (5T OTA, two-stage Miller), netlist validator, logic synthesis (20 cells), ngspice driver, optimizers. New here: DC-servo testbench and corner forking.

Key mechanisms

The compiler as a test suite. The architect's JSON plan is checked deterministically:

- Each amplifier spec must parse to targets with units.
- Gain must be ≤ 78 dB, the measured sky130 two-stage ceiling with headroom.
- GBW must be ≤ 25 MHz, and load must be 0.1–20 pF.
- Logic cells must exist; there must be 1–2 analog and 1–3 digital blocks.

Errors are phrased so the model can act on them. In the mock live run the architect asked for 120 dB, was told "beyond the fab's two-stage op-amp on sky130 (max ~78dB)", and repaired the plan in one round.

The sizing race. Two racers, each with its own simulation counter against the same models:

- The engine's LLM sizing agent, adapted to Token Factory through a small client shim.
- Differential evolution.

The first to meet every target stops the other. If a single-stage OTA cannot reach the gain, the block escalates to the two-stage Miller op-amp.

Forks. A sized design is frozen and content-hashed. Each stress condition renders its own deck from the frozen values (process-corner include files swapped, supply, temperature, load, common mode) and runs in its own ngspice process. A failing run is a leaf in the log, never a change to the design.

Provenance. Every agent reports whether it ran live on Nemotron, and if not, why. The UI shows a `LIVE NEMOTRON · ACTIVE` or `FALLBACK` banner and per-agent tags; the datasheet carries a provenance table.

Replay and static demo.

- `tapeout --record` saves a run's event stream. The server replays it time-compressed (about 25 s), labelled REPLAY, with every trace still clickable.
- `export_static.py` inlines a recording into a single HTML file for GitHub Pages: no cold start, no credits.

5. The falsification engine in detail

Stress knobs and the operating envelope

Knob	Adversary may use	Envelope (the promise)
Process corner	tt, ss, ff, sf, fs (sky130 foundry corner models)	all five
Supply	1.5 – 2.0 V	1.71 – 1.89 V (±5%)
Temperature	−55 – 150 °C	−40 – 85 °C
Input common mode	0.1 – 0.9 · VDD	0.4 – 0.6 · VDD
Load capacitance	1 – 10 × nominal	nominal

A break **inside** the envelope is a design bug, and the design team must repair it. A break **outside** it is a limit, published with its boundary and the run that proves it.

One round of attack

1. **Hypotheses.** Nemotron Ultra receives the topology, every sized parameter and each claim's margin. It returns 4–6 hypotheses, each with:
   - a claim
   - a one-sentence circuit mechanism
   - fixed knobs
   - one axis to push toward an extreme

   Out-of-range values are clipped; unknown claims and axes are discarded.
2. **Boundary search.** Each hypothesis is simulated at its fixed point, then at the extreme of its axis. If the claim breaks there, the axis is bisected 5 times back toward nominal to locate the boundary. This takes 2–7 simulations per hypothesis.
3. **Corner grid.** All 5 corners × 3 supplies × 3 temperatures (45 points) across the envelope. If the grid finds a violation the hypotheses missed, the report says so.
4. **Robustness table.** For each claim: typical value, worst case anywhere in the envelope (from hypotheses or grid), status, and limits found outside.

Repair and re-attack

- For each broken claim, the single worst in-envelope condition is added to a **regression set** that only grows.
- The design is re-sized against nominal plus every condition in that set. The score is the worst case across them, and each candidate's conditions are simulated in parallel.
- Differential evolution is seeded with the broken design plus jittered copies. The Nemotron lane starts by evaluating the broken design, the way an engineer edits a schematic rather than starting over.
- The repaired design is attacked again: the old hypotheses as regressions, plus 2–3 new ones the model writes for the new sizing.
- Up to 2 repairs are made per block.

Shmoo plot

After the campaign, the original and final designs are each simulated over 6 supplies (1.5, 1.6, 1.71, 1.8, 1.89, 2.0 V) × 4 temperatures (−40, 27, 85, 125 °C) × 5 corners. That is 120 cells, of which 45 are reused from the corner grid. Each cell is marked with the claims that failed there.

6. Development trial log

A chronological record of what was built, what was tried, and what broke. The most important entries are bugs that the falsification loop itself exposed.

Trial 1: baseline engine on sky130

The cadence_auto engine sized a 40 dB / 5 MHz / 200 µW op-amp on sky130 in 5 simulations (0.9 s), and proved a full adder in 3.3 s. With the original testbench, gain ceilings were measured by sizing probes:

Request (sky130, original testbench)	Topology	Met?	Sims	Measured gain
46 dB, 10 MHz, 100 µW, 1 pF	5T OTA	yes	`205`	`46.3 dB`
60 dB, 5 MHz, 200 µW, 2 pF	Miller	yes	`325`	`62.1 dB`
70 dB, 2 MHz, 300 µW, 5 pF	Miller	no	`800`	`64.5 dB (best)`
55 dB, 1 MHz, 50 µW, 10 pF	Miller	yes	`279`	`64.9 dB`

Trial 2: the testbench lied (bug #1)

Before building the falsifier, a 60 dB design was swept across corners. A 5% supply change or 85 °C "collapsed" its gain:

Corner	Condition	Gain	Output headroom
tt	1.8 V, 27 °C (nominal)	`63.2 dB`	22%
tt	1.71 V, 85 °C	`27.8 dB`	2%
tt	1.89 V, −40 °C	`44.8 dB`	6%
fs	1.8 V, 27 °C	`18.7 dB`	1%

Diagnosis: the amplifier was measured open-loop with no DC feedback, so a few millivolts of offset railed its output. A real part sits in a feedback loop and never shows this. The falsifier would have published dozens of fake failures.

Fix: a DC servo. A 1 GH inductor into a 1 F capacitor feeds the output back to the inverting input. This pins the operating point but is open at every AC frequency measured, so the AC measurement stays open-loop. The inverting input is per-topology: `vinn` for the OTA, `vinp` for the Miller.

With the servo, the same sweep showed physically sensible failures: phase margin 55.7° → 43.5° at 85 °C and → 32° at 4× load, and the PMOS input pair cutting off at Vcm = 0.8·VDD. Re-probing showed the real two-stage gain ceiling on sky130 is about **82 dB**, not 65 dB. The architect's limits were raised from 66 to 78 dB.

Trial 3: first full arms race (offline)

Round 1 broke claims, repair 1, round 2 broke more, repair 2, round 3 clean. That was 185 stress simulations in 3½ minutes. The deterministic seeds made it reproducible, which exposed the next bug.

Trial 4: repairs targeted the wrong corners (bug #2)

Repair 1 sized against `tt/1.71V/−40C` and `tt/1.8V/85C`. Those were the *first three grid failures*, not the worst. Meanwhile GBW had collapsed to 76 kHz at sf/1.71 V/−40 °C and PM to 23.2° at fs/1.89 V/85 °C. Repair conditions now come from each broken claim's worst in-envelope run.

Trial 5: is it the budget or the physics?

With the right corners, repair failed within 2 × 300 candidates. An experiment settled whether a fix existed at all. Differential evolution against nominal + 2 worst corners, and against nominal + 4 corners, found a sizing meeting every condition in **481 candidates** (63.4 dB, PM 63°, GBW 5.9 MHz). So it was budget, not physics. Repairs now get 500 candidates and start from the broken design: generation 0 holds the design plus jittered copies.

Trial 6: a repair reopened an old failure (bug #3)

Repair 2 sized against round 2's corners only, and GBW broke again at the round-1 corner. The final re-attack caught it, so the published result was correctly *derated*. Fix: every counterexample ever found stays in a regression set that every later repair must also satisfy. With that, the run converged: round 3 found nothing inside the envelope.

Trial 7: the live path against a protocol-faithful mock

No API key was available in the development environment. A mock Token Factory was written that speaks the real dialect:

- `/v1/models`
- streamed `reasoning_content` deltas
- inline `<think>` tags split across chunks
- an empty-`choices` usage chunk
- `[DONE]`
- injectable disconnects and malformed chunks

Its first run found three bugs offline mode could never reach:

Bug	Effect	Fix
`<think>` tag split across stream chunks	The architect's whole answer was swallowed as "thinking", so the run silently fell back to the reference plan. A live run would have looked like offline mode.	A streaming splitter that holds back partial tags; tested at every split position
Cached re-proposals are free	The sizing lane made **8,884 calls** (≈ $1.17) in one run	Calls capped at 2× the lane's simulation budget (32 calls on the same run)
Empty datasheet reply	Counted as success	Treated as a failure

A dropped stream is now a retried error, not a short answer. Model ids resolve case-insensitively against `/v1/models`. Every agent now reports provenance.

Trial 8: container and CI

A local image build was blocked by the sandbox network (the proxy refuses plain-HTTP apt). The image is therefore built in GitHub Actions instead. Its self-check passed in CI: a half adder proved and a slow-corner 85 °C fork simulated inside the image. The first smoke test failed only because Docker rejects a one-character container name. The fix was pushed and CI went green.

Trial 9: shmoo, commercial reference, static demo, clean room

- The shmoo plot was added. A 6 × 6 × 5 grid took 61 s, so it was trimmed to 6 × 4 × 5, reusing the grid's runs.
- The Tavily commercial reference was added with verbatim verification. In the mock run, the model's invented "140 dB" was discarded.

- The static GitHub Pages replay was built and tested: 25 s, zero API calls.
- A clean-room clone passed all 340 tests (the count at that commit) in 118 s.

Trial 10: re-running the evidence

To back the claim that every number is traceable, `tapeout.verify` re-simulates every stored deck of a recording. Its first pass reported 2 of 83 runs as mismatched. Both were at extreme common-mode points where the amplifier has no unity-gain crossing, so phase margin is undefined. The run had recorded this as `null` (JSON has no NaN), and the re-run produced NaN. With "undefined" compared as equal on both sides, **83 of 83 runs and 413 of 413 measurements reproduce**. Tests confirm a doctored measurement or deck still fails.

7. Results: the reference run

Offline mode: the built-in reference plan and textbook hypotheses stand in for Nemotron, and every simulation is real. Prompt: "a heart-rate (PPG) monitor front-end for a fitness band". Recorded 29 Oct 2026, final code. Total: 399 simulations, 252 s, 68 transistors.

Plan and nominal sizing

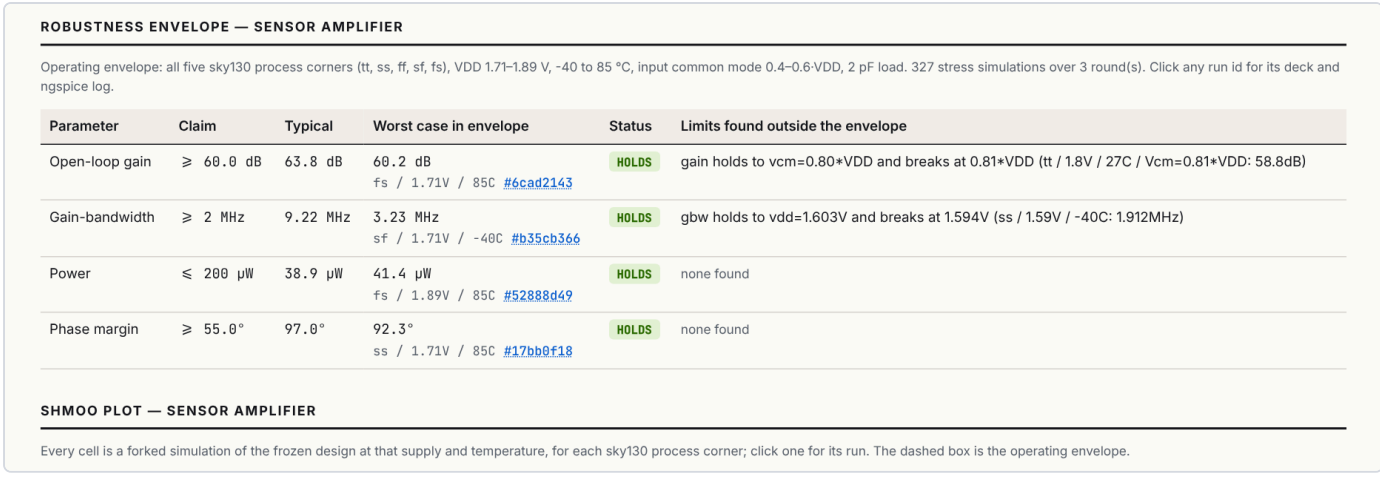
Block	Kind	Specification / cell	Result
Sensor amplifier	analog, two-stage Miller (13 parameters)	≥ 60 dB gain, ≥ 2 MHz GBW, ≤ 200 µW, 2 pF load; implied PM ≥ 55°, headroom ≥ 20%	sized by differential evolution in 70 sims
Level comparator	digital, comparator1	34 transistors	4/4 truth-table rows proved
Debounce voter	digital, majority3	26 transistors	8/8 truth-table rows proved

Round by round

Round	Design	Targeted + grid sims	Grid violations (of 45)	What happened
1	`5507a6a1`	`13 + 39`	GBW 19, PM 23	`BROKEN` PM 47.4° at ff, 41.8° at fs; GBW 202 kHz at ss/−40 °C. Worst in envelope: **PM 23.2°** (fs/1.89 V/85 °C), **GBW 76 kHz** (sf/1.71 V/−40 °C)
repair 1	Re-sized against nominal + sf/1.71 V/−40 °C + fs/1.89 V/85 °C			
2	`fa916125`	`26 + 39`	gain 2, PM 3	PM and GBW now hold at those corners. `FOUND BY GRID` gain 59.7 dB (ff/1.71 V/85 °C) and PM 54.6° (ss/1.8 V/85 °C), which the hypotheses missed
repair 2	Re-sized against nominal + all four corners that ever broke a claim			
3	`998c38f8`	`21 + 39`	none	`NOTHING BREAKS IN ENVELOPE`. Limits published outside it.

Final robustness envelope

Claim	Typical	Worst in envelope	Status	Limit found outside
Gain ≥ 60 dB	`63.8 dB`	`60.2 dB (fs/1.71 V/85 °C)`	`HOLDS`	breaks at Vcm 0.81·VDD (58.8 dB)
GBW ≥ 2 MHz	`9.22 MHz`	`3.23 MHz (sf/1.71 V/−40 °C)`	`HOLDS`	breaks below 1.594 V at ss/−40 °C (1.91 MHz)
Power ≤ 200 µW	`38.9 µW`	`41.4 µW (fs/1.89 V/85 °C)`	`HOLDS`	holds to 2.0 V at ff/−40 °C
PM ≥ 55°	`97.0°`	`92.3° (ss/1.71 V/85 °C)`	`HOLDS`	holds to 6× load at ff, and to 150 °C at fs



ROBUSTNESS ENVELOPE — SENSOR AMPLIFIER

Operating envelope: all five sky130 process corners (tt, ss, ff, sf, fs), VDD 1.71–1.89 V, -40 to 85 °C, input common mode 0.4–0.6·VDD, 2 pF load. 327 stress simulations over 3 round(s). Click any run id for its deck and ngspice log.

Parameter	Claim	Typical	Worst case in envelope	Status	Limits found outside the envelope
Open-loop gain	⩾ 60.0 dB	63.8 dB	60.2 dB fs / 1.71V / 85C #6cad2143	HOLDS	gain holds to vcm=0.80*VDD and breaks at 0.81*VDD (tt / 1.8V / 27C / Vcm=0.81*VDD: 58.8dB)
Gain-bandwidth	⩾ 2 MHz	9.22 MHz	3.23 MHz sf / 1.71V / -40C #b35cb366	HOLDS	gbw holds to vdd=1.603V and breaks at 1.594V (ss / 1.59V / -40C: 1.912MHz)
Power	⩽ 200 µW	38.9 µW	41.4 µW fs / 1.89V / 85C #52888d49	HOLDS	none found
Phase margin	⩾ 55.0°	97.0°	92.3° ss / 1.71V / 85C #17bb0f18	HOLDS	none found

SHMOO PLOT — SENSOR AMPLIFIER

Every cell is a forked simulation of the frozen design at that supply and temperature, for each sky130 process corner; click one for its run. The dashed box is the operating envelope.

The robustness table as rendered on the datasheet. Each worst case links to its run.

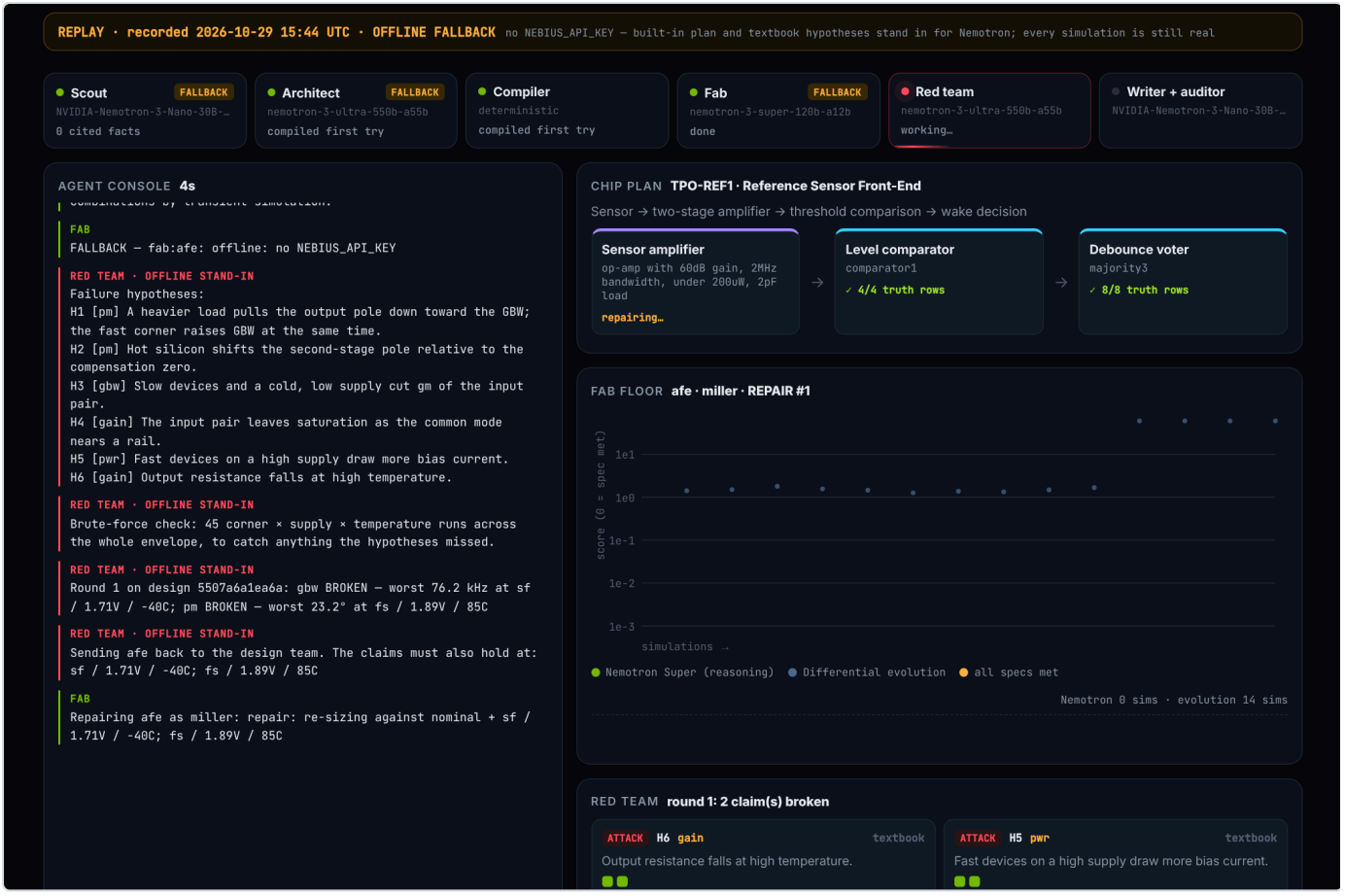
Shmoo: first sizing vs. after repair

Design	Cells	Failing cells	Failing inside envelope
as first sized (5507a6a1)	120	113	**42**
after 2 repairs (998c38f8)	120	16	**0**

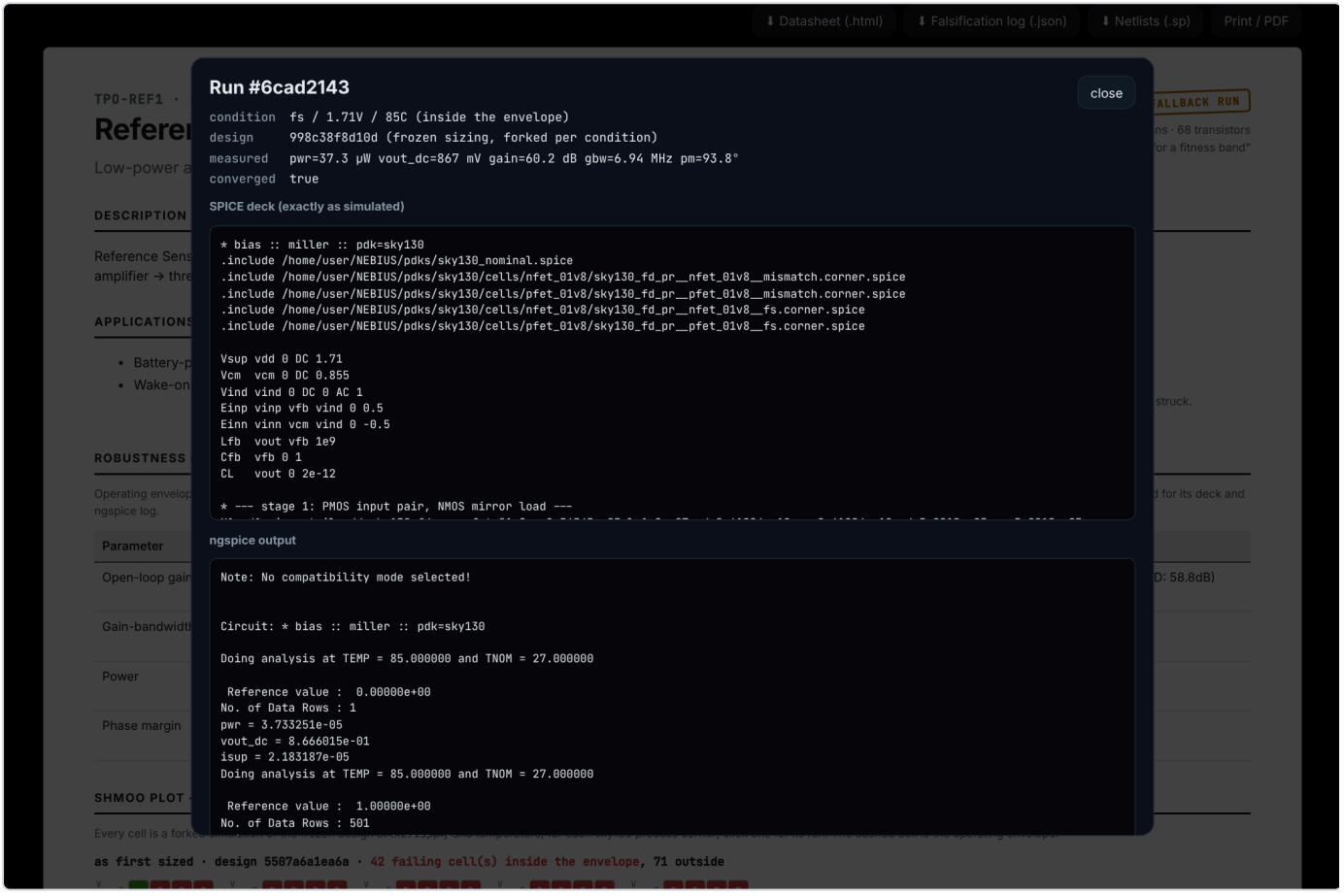
All 16 remaining failures lie outside the envelope:

- gain, at 85–125 °C with low supply on the ff and fs corners
- bandwidth, at −40 °C below 1.71 V on the tt, ss and sf corners

These are the published limits.



The live UI during the reference run, replayed. The amber banner and FALLBACK tags declare that offline stand-ins did the agents' work. The red team's round-1 verdict and the repair conditions are in the console.



Clicking any run id opens its exact SPICE deck (corner include files, supply, servo loop) and the raw ngspice output.

8. Verification

Automated tests: 346

Suite	Tests	Covers
`test_logic.py`	106	All 20 logic cells, synthesis and truth tables
`test_live_path.py`	50	Live Token Factory path against the mock: `<think>` splitting at every position, discovery and casing, streaming and usage, malformed chunk, dropped stream, architect repair, writer audit rewrite, sizing call cap, scout citations, commercial reference verification
`test_specparse.py`	47	Spec parsing, units, mW vs MHz, implied constraints
`test_sky130.py`	32	Real foundry-model simulations
`test_netlist.py` · `test_topology.py` · `test_scoring.py` · `test_sim.py` · `test_agent.py`	85	Validator, topologies, scoring, ngspice driver, LLM sizing agent
`test_tapeout.py`	16	Compiler accept/reject reasons, auditor, envelope classification, hypothesis clipping, servo, fork immutability, worst-run repair targeting, shmoo, offline check
`test_replay.py`	4	Replay listing, labelling, streaming, unsafe names rejected
`test_verify.py`	6	Independent re-simulation: path rewriting, a recorded deck reproduces, a doctored number and a doctored deck are caught

Independent re-simulation

`python -m tapeout.verify <recording>` re-runs every SPICE deck a recorded run kept on the checker's own ngspice and sky130 models, and compares each recorded measurement: gain, GBW, power, output DC level and phase margin. It trusts nothing from the run except the deck text.

Reference run: all **83** stored decks reproduced. **413 of 413** recorded measurements matched within 10^{-3} relative. CI repeats this on GitHub's machines on every push.

Continuous integration

Every push runs two jobs on GitHub Actions:

- **test:** installs ngspice, fetches sky130 and runs the suite.
- **image:** builds the Docker image and smoke-tests the running container. The image build itself fails unless a half adder is proved and an ss / 1.71 V / 85 °C fork simulates inside it.

Both were green on code commit `a07f5d8`. The test job now also re-simulates the bundled recording.

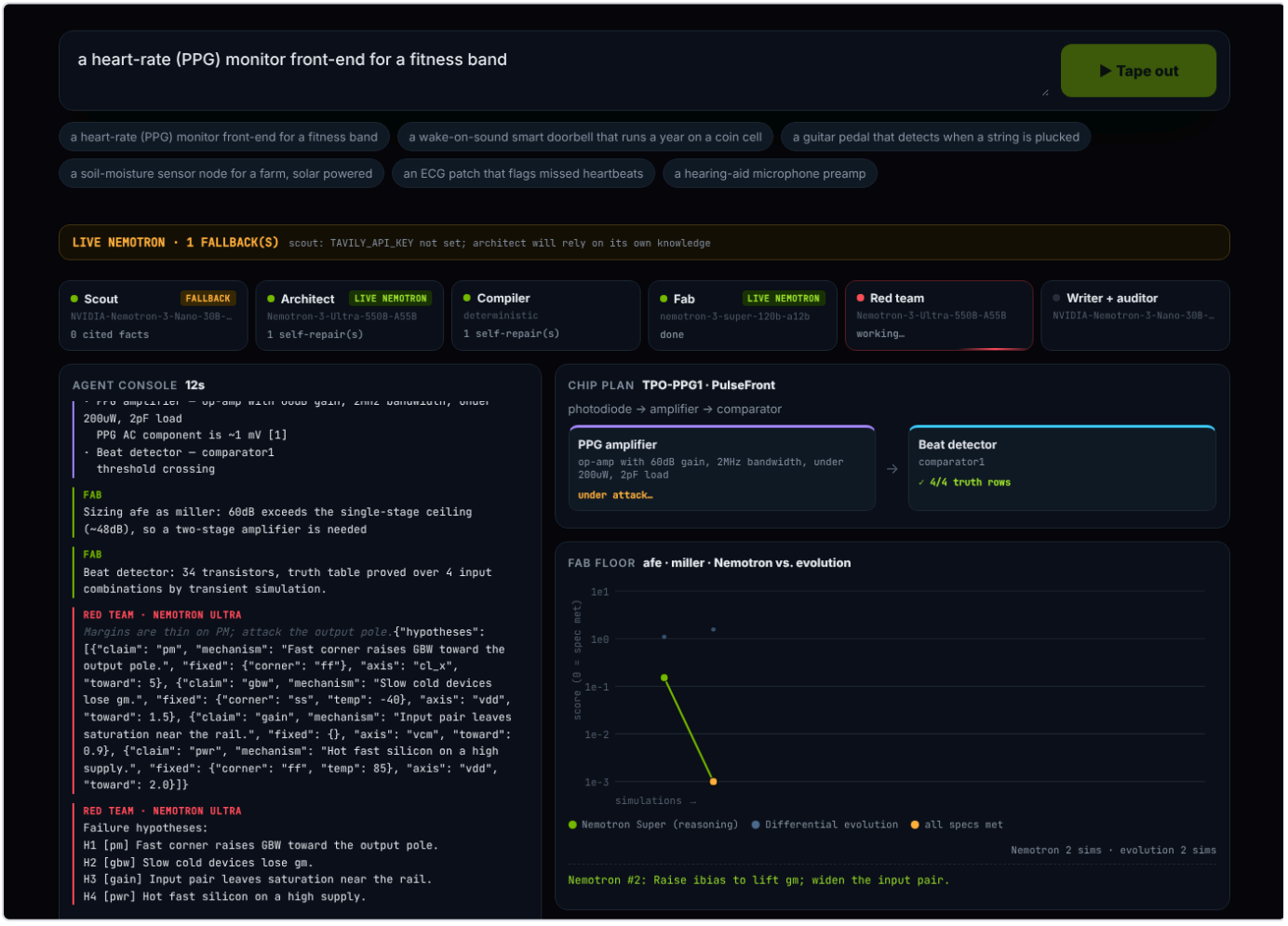
Clean room

A fresh clone from GitHub, a new virtualenv and only the README's commands, with no API keys. The offline check reported `SIMULATION PATH READY` and all 340 tests passed, in **118 s** total. ngspice was already installed system-wide on that machine. The full record is in `docs/CLEANROOM.md`.

Mock live run (every agent through the Token Factory protocol)

Agent	Status	Detail
scout	LIVE	1 cited fact from 1 Tavily result
scout: reference	LIVE	2 figures verified verbatim against the retrieved page; 1 discarded (the model's invented "140 dB")
architect	LIVE	plan compiled after 1 repair round (120 dB rejected)
fab	LIVE	Nemotron lane proposed sizings and won the race
red team	LIVE	5 of 5 hypotheses from the model, including an adaptive one after repair
writer	LIVE	prose audited; 1 rewrite forced by unsupported figures

This run used a mock server, not the real Nemotron models. It proves every byte of the integration, not the models' reasoning quality. 776 simulations, 204 s. Calls: Ultra 3, Super 30, Nano 5.



The UI against the mock Token Factory: a LIVE NEMOTRON banner naming its one declared fallback (scout, no Tavily key in that run), LIVE tags on each agent, and Nemotron's sizing reasoning under the race chart.

9. Limitations, and what is not yet verified

Not yet run against the real Nebius Token Factory. No API key was available during development. The live path is exercised end to end against a protocol-faithful mock, so integration bugs are covered. The real models' reasoning quality, latency and cost are not. Verify with `python -m tapeout.check` before any demo.

- **Model ids.** Nemotron 3 ids could not be confirmed from Nebius's own docs. The app resolves whatever `/v1/models` lists and fails with the available list if a tier is missing.
- **Two amplifier topologies** (5T OTA, two-stage Miller) **and 20 logic cells.** The architect works inside that menu and is told so.
- **Attacked metrics:** gain, GBW, phase margin and power. There is no noise, slew-rate, PSRR or Monte Carlo mismatch analysis; the sky130 statistical parameters are zeroed at the nominal corner.
- **"Holds" means sampled, not proven.** It holds at every grid point, shmoo cell and adversarial probe. That is strong evidence, not a formal proof.
- **Forks are isolated ngspice processes in one container,** not Nebius Sandboxes or Serverless Jobs (a natural next step).
- **The commercial reference is a published datasheet,** cited and verified verbatim, but never simulated here.
- **Container.** Built and smoke-tested in CI only, never locally (the sandbox network blocked it).

10. Running, deploying and demonstrating it

Repository: https://github.com/Amritha902/NEBIUS

```
git clone https://github.com/Amritha902/NEBIUS.git && cd NEBIUS
sudo apt install ngspice            # macOS: brew install ngspice
scripts/fetch_pdk.sh sky130
python3 -m venv .venv && .venv/bin/pip install -e '.[dev]'
.venv/bin/python -m tapeout.check --offline     # no keys: simulator path
.venv/bin/python -m tapeout.verify web/replays/ppg-offline.jsonl   # re-run the evidence
export NEBIUS_API_KEY=... TAVILY_API_KEY=...
.venv/bin/python -m tapeout.check               # must print LIVE PATH READY
.venv/bin/uvicorn web.server:app --port 8010    # http://localhost:8010
.venv/bin/tapeout "a heart-rate (PPG) monitor front-end for a fitness band" --record
web/replays/ppg-live.jsonl
python scripts/export_static.py                 # static replay → docs/demo/
```

- **Static demo:** GitHub Settings → Pages → `main` / `/docs`, which serves https://amritha902.github.io/NEBIUS/demo/
- **Live demo:** `render.yaml` (Docker), with both keys set as environment variables.
- **Video script:** `docs/DEMO.md`. Devpost text: `docs/DEVPOST.md`. Setup guide: `Tapeout-Setup-Guide.pdf`.

11. Appendix

Commit history

Commit	Change
`da0f5ff`	Tapeout: Nemotron agents design a chip, then a red team breaks its datasheet (engine import, DC servo, agents, falsifier, repair with regressions, UI, Docker)
`abcf4fb`	Live path testable and every fallback visible (mock, three integration bugs, provenance, model check, CI image job)
`0086b72`	CI: valid smoke-test container name
`d4347f9`	Instant replay of recorded runs; Devpost write-up
`a07f5d8`	Shmoo plots, Tavily-verified commercial reference, static demo, stronger check
`8ef5b74`	Clean-room verification record
`3ff744c`	Demo script
—	LICENSE kept pure MIT so GitHub detects it; PDK note moved to NOTICE
`5209cd1`	Project report and setup guide added as PDFs
`520f2e9`	`tapeout.verify`: independent re-simulation of a run's evidence (83 decks, 413/413 measurements)

Full history: https://github.com/Amritha902/NEBIUS/commits/main

Glossary

PDK	Process design kit: the foundry's device models. Here, SkyWater sky130 (130 nm, open source).
Corner	A manufacturing extreme. tt typical; ss/ff slow/fast for both device types; sf/fs mixed.
GBW	Gain-bandwidth: the frequency where the amplifier's gain falls to 1.
Phase margin (PM)	How far from oscillation the amplifier is when wrapped in feedback; below ~45–55° it rings or oscillates.
Shmoo plot	Pass/fail grid over operating conditions, as drawn by chip test floors.
DC servo	A feedback loop active only at DC, which sets the operating point while leaving the AC measurement open-loop.
Fork	One simulation of a frozen design under one stress condition, in its own process.
Envelope	The operating range the datasheet promises.

Tapeout is MIT-licensed. SkyWater sky130 models are Apache-2.0 and fetched at build time, not vendored. The circuit engine in `bias/` comes from the author's earlier cadence_auto project; everything in `tapeout/` and `web/`, the DC servo and the corner forking were built during the submission period.